

Project Report Documentation

Team ID:

Date: March 7, 2026

1. PROJECT TITLE :-

Visualization Tool for Electric Vehicle Charge and Range Analysis

2. PROJECT DESCRIPTION :-

- **System Overview:** An end-to-end EV analytics and web integration project that visualizes multidimensional charging and battery data to optimize infrastructure planning, fleet management, and consumer decision-making.
- **Problem Domain:** Electric Vehicle Charging Analytics and Range Optimization.
- **Core Technology Used:** Tableau Desktop/Public, Python, Flask Web Framework, HTML/CSS.
- **Key Capabilities:** Interactive dashboards, charging pattern heatmaps, battery health vs. range scatter plots, comparative EV model efficiency, and seamless web embedding.
- **Target Users:** City Energy Planners, Fleet Managers, and EV Consumers.
- **Real-world Relevance:** Addresses the growing need for sustainable EV adoption by providing actionable insights into charging demand, battery performance, and model efficiency.

3. APPLICATION SCENARIOS / USE CASES :-

Charging Pattern and Usage Analysis

A city energy planner analyzes EV charging data such as charging time, duration, and power usage. The dashboard identifies peak charging hours and areas with high electricity demand, helping planners manage grid stability and develop charging infrastructure.

Battery Performance vs Driving Range

An EV fleet manager monitors battery health and driving range of vehicles. By comparing predicted and actual range, the manager can detect underperforming batteries and schedule maintenance.

Comparative EV Model Efficiency Dashboard

Consumers can compare different EV models based on efficiency, cost per kilometer, and environmental impact to make informed purchasing decisions.

4. TECHNICAL ARCHITECTURE OVERVIEW :-

Data / Request Flow

Data Layer

EV dataset stored in a CSV file containing vehicle information such as battery capacity, charging time, energy consumption, and driving range.

Analytics Layer

Tableau connects to the dataset and creates interactive dashboards and visualizations.

Cloud Layer

The Tableau dashboard is published on Tableau Public to generate an embed link.

Backend Web Layer

A Flask web application serves the dashboard to users.

Frontend Layer

HTML and CSS create the user interface and embed the Tableau dashboard using iframe.

5. PREREQUISITES:-

5.1 Software Requirements

Development Tools:

- Tableau Desktop
- VS Code or PyCharm

Runtime Environment:

- Python 3.8+
- Web browser (Chrome, Edge, Firefox)

5.2 Libraries / Frameworks

- Flask
- Tableau JavaScript API

6. PRIOR KNOWLEDGE REQUIRED :-

- Concepts required before implementation:
- Basic knowledge of Data Analytics
- Understanding of Electric Vehicle concepts
- Tableau dashboard creation
- Basic Python programming
- Basic HTML and web development

7. PROJECT OBJECTIVES :-

Technical Objective

Analyze EV data such as battery performance, charging patterns, and vehicle efficiency.

Performance Objective

Create interactive dashboards that quickly respond to filters and display insights clearly.

Deployment Objective

Integrate the Tableau dashboard into a web interface using Flask for easy access.

8. SYSTEM WORKFLOW :-

Step-by-step execution flow: -

- The EV dataset is collected and stored in CSV format.
- Data cleaning and preprocessing are performed.
- Data is connected to Tableau for visualization.
- Dashboards and stories are created.
- The dashboard is published to Tableau Public.
- The dashboard is embedded into a Flask web application.
- Users interact with filters and analyze EV insights.

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN :-

Objective: Data Collection

- Collect EV dataset containing battery capacity, charging duration, and range.
- Connect dataset with Tableau.
- Verify correct data types for analysis.

10. MILESTONE 2: SYSTEM DESIGN:-

Dashboard Layout

- KPI summary section
- Charging pattern graphs
- Battery performance charts
- EV model comparison charts

Web UI Design

- Sidebar navigation
- Dashboard embedding
- Responsive layout

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT :-

11.1 Data Preparation

- Clean EV dataset
- Convert numeric fields like battery capacity and energy consumption
- Create calculated fields

Example calculation:

Range Efficiency = Distance Travelled / Energy Consumed

11.2 Data Visualizations

Created 4 visualizations:

1. **Bar Chart**
Charging frequency by time of day.
2. **Line Chart**
Battery charge vs driving range.
3. **Heatmap**
EV charging demand by location and time.
4. **Scatter Plot**
Battery health vs vehicle efficiency.

11.3 Dashboard & Story Creation

Dashboard

Combined multiple charts into one interactive dashboard with filters.

Story

A 3-step story explaining:

1. Charging behavior overview
2. Battery performance analysis
3. EV model efficiency comparison

12.MILESTONE 4: INTEGRATION & OPTIMIZATION

Web Integration

- Developed Flask application
- Embedded Tableau dashboards in HTML page
- Implemented navigation between dashboard views

13. MILESTONE 5: TESTING & VALIDATION

13.1 Data Validation

13.2 Dashboard Functionality Testing

13.3 Visualization Accuracy Testing

13.4 Performance Testing

13.5 Result Validation

14. DEPLOYMENT

- **Deployment Architecture:** Local Flask Web Server + Cloud Hosted Tableau Dashboard.
- **Hosting Platform:** Tableau Public (for visualization hosting) and local environment (for Flask web app).
- **Deployment Steps:** 1. Publish Workbook to Tableau Public.
2. Copy Embed Code.
3. Insert into Flask index.html. 4. Run python app.py.

15. PROJECT STRUCTURE :-

EV-Charge-Range-Analysis/

```
|
|
|— data/
|   |— ev_dataset.csv
|
|
|— tableau_dashboard/
|   |— ev_dashboard.twbx
|
|
|— images/
|   |— price_analysis.png
```

```
| |— efficiency_chart.png
| |— body_style_chart.png
| |— brand_models.png
| |— powertrain_tree_map.png
|
|— report/
| |— ev_project_report.pdf
|
|— presentation/
| |— ev_project_presentation.pptx
|
|— video/
| |— ev_project_demo.mp4
|
|— README.md
```

16. RESULTS :-

The system provides a web-based visualization platform where users can analyze EV charging patterns, battery performance, and vehicle efficiency through interactive dashboards.

[Screenshot Placeholder]

17. ADVANTAGES & LIMITATIONS :-

Advantages:-

- Centralized access via web browser.
- Intuitive dashboards with no-code interactivity.
- Supports multiple user roles (planner, manager, consumer).
- Enables proactive decision-making (battery replacement, infrastructure planning).

Limitations:-

- Relies on static CSV extracts; real-time updates require manual refresh.
- Tableau Public hosting may limit enterprise-level security.
- Predictive analytics are basic; advanced ML integration is needed for deeper insights.

18. FUTURE ENHANCEMENTS :-

- **Live Database Integration:** Connect Tableau directly to a live SQL database (e.g., Snowflake) instead of a CSV for real-time updates.
- **Advanced Flask Integration:** Implement Flask-Login to create secure user authentication before viewing the dashboard.

19. CONCLUSION:-

This project successfully integrates the analytical power of Tableau with the flexibility of a Python Flask web application. By completing the end-to-end process—from raw data collection and preparation to complex visualization, story building, and final web deployment—the solution directly addresses the business need for optimized, data-driven retail merchandise placement

20. APPENDIX:-

- **Project Documentation:** Step-by-step project development procedures are stored in the project's README.md.
- **Demonstration Video:** https://drive.google.com/file/d/1IJsAaWuX2mwM-yICLn2xAHl9qf3e6KFc/view?usp=drive_link
- **GitHub Repository Link:** *[Insert GitHub URL here]*
- **Live Dashboard Link:** https://public.tableau.com/views/EVVisualization2/EVDashboard?:language=en-US&:sid=&:redirect=auth&:display_count=n&:origin=viz_share_link
- **A.1 Source Code: Flask Backend (app.py)**

```
from flask import Flask, render_template
```

```
app = Flask(__name__)
```

```
@app.route("/")  
def home():
```

```
return render_template("index.html")
```

```
if _name_ == "_main_":  
    app.run(debug=True)
```

A.2 Source Code: Frontend UI (index.html)

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>EV Dashboard</title>  
    <style>  
        /* This removes the default white border/margin around the page */  
        body, html {  
            margin: 0;  
            padding: 0;  
            height: 100%;  
            width: 100%;  
            overflow: hidden; /* Prevents double scrollbars */  
        }  
  
        /* This forces the iframe to fill the entire screen */  
        iframe {  
            border: none;  
            width: 100vw;  
            height: 100vh;  
        }  
    </style>  
</head>  
<body>  
  
    <iframe  
        src="https://public.tableau.com/views/EVVisualization2/EVDashboard?:embed=yes&:showVizHome=no"  
        allowfullscreen>  
    </iframe>  
  
</body>  
</html>
```